

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, clas
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline
```

```
In [ ]: df = pd.read_csv("Social_Network_Ads.csv")
```

```
In [ ]: df
```

```
In [ ]: df.shape
```

```
In [ ]: df.info()
```

```
In [ ]: df.describe()
```

```
In [ ]: df.isna().sum()
```

```
In [ ]: histplot = sns.histplot(df['Age'], kde=True, bins=10, color='red', alpha=
for i in histplot.containers:
    histplot.bar_label(i,)
plt.show()
```

```
In [ ]: histplot = sns.histplot(df['EstimatedSalary'], kde=True, bins=10, color='
for i in histplot.containers:
    histplot.bar_label(i,)
plt.show()
```

```
In [ ]: df["Gender"].value_counts()
```

```
In [ ]: def gender_encoder(value):
    if (value == "Male"):
        return 1
    elif (value == "Female"):
        return 0
    else:
        return -1
```

```
In [ ]: df["Gender"] = df["Gender"].apply(gender_encoder)
df
```

```
In [ ]: df["Purchased"].value_counts()
```

```
In [ ]: sns.pairplot(df, hue='Purchased', palette='bwr')
plt.show()
```

```
In [ ]: sns.heatmap(df.corr(), annot=True)
plt.show()

In [ ]: x = df[["User ID", "Gender", "Age", "EstimatedSalary"]]
y = df["Purchased"]

In [ ]: scaler = StandardScaler()
x = scaler.fit_transform(x)

In [ ]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,

In [ ]: x_train.shape, x_test.shape, y_train.shape, y_test.shape

In [ ]: model = LogisticRegression()

In [ ]: model.fit(x_train, y_train)

In [ ]: y_pred = model.predict(x_test)
y_pred

In [ ]: model.score(x_train, y_train)

In [ ]: model.score(x, y)

In [ ]: cm = confusion_matrix(y_test, y_pred)
print(cm)

In [ ]: disp=ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.clas
disp.plot()
plt.show()

In [ ]: print(f"TN value is {cm[0][0]}")
print(f"FP value is {cm[0][1]}")
print(f"FN value is {cm[1][0]}")
print(f"TP value is {cm[1][1]}")

In [ ]: print(f"Accuracy score is {accuracy_score(y_test, y_pred)}")
print(f"Error rate is {1-accuracy_score(y_test, y_pred)}")
print(f"Precision score is {precision_score(y_test, y_pred)}")
print(f"Recall score is {recall_score(y_test, y_pred)}")

In [ ]: print(classification_report(y_test, y_pred))

In [ ]:
```